Protocolo experimental: persistência local e funcionamento offline

FixTag · Tema 3 · Hugo Carvalho · 27 de outubro de 2026

Repositório: https://github.com/0xmaiev/FixTag, branch `entrega-10-27`. A aplicação Android está na pasta `android/`.

> Este protocolo descreve cinco experiências que outro grupo pode repetir com o código do repositório, sem servidor e sem credenciais: a variante `demo` da aplicação funciona apenas com dados locais.

1. Enquadramento teórico

1.1 Porque é que a persistência local importa

Uma aplicação móvel não pode assumir que tem rede. O FixTag regista avarias em máquinas industriais, e quem as reporta está muitas vezes em pavilhões ou caves sem cobertura. Uma aplicação *offline-first* trata a base de dados local como a fonte de verdade: a interface lê e escreve sempre no dispositivo, e a sincronização com o servidor acontece depois, em segundo plano.

O Android oferece vários mecanismos de persistência, cada um adequado a um tipo de dado. Todos guardam os dados no armazenamento privado da aplicação (`/data/data/<pacote>/`), a que as outras aplicações não têm acesso e que é apagado quando a aplicação é desinstalada.

Mecanismo	Para quê	Diretório	No FixTag
SharedPreferences	pares chave-valor simples	`shared_prefs/`	não usado
DataStore	chave-valor (Preferences) ou objetos tipados (Proto)	`files/datastore/`	sessão (perfil escolhido)
Ficheiros internos	dados binários: fotos, áudio, documentos	`files/`, `cache/`	anexos
SQLite / Room	dados estruturados e relacionados	`databases/`	organizações, máquinas, ocorrências

1.2 Pares chave-valor: SharedPreferences e DataStore

As `SharedPreferences` guardam pares chave-valor num ficheiro XML. A API é síncrona: a primeira leitura carrega o ficheiro na thread que a pede, e `apply()`, apesar de gravar em segundo plano, pode bloquear a thread principal quando a atividade é pausada, o que provoca ANR (*Application Not Responding*). Também não sinalizam erros de escrita nem oferecem transações.

O **DataStore** (Jetpack) substitui-as. Expõe os dados como um `Flow` de Kotlin, lê e escreve fora da thread principal, faz cada atualização com `edit { }` de forma atómica (ler, modificar e escrever numa só operação) e propaga falhas como `IOException`. Tem duas variantes: *Preferences DataStore*, de chave-valor e sem esquema, e *Proto DataStore*, com objetos tipados definidos em Protocol Buffers. Deve existir uma única instância por ficheiro: duas instâncias ativas sobre o mesmo ficheiro lançam uma exceção. Na variante `demo` do FixTag, o DataStore guarda o perfil escolhido (chave `current_user_id`) no ficheiro `files/datastore/session.preferences_pb`.

1.3 Ficheiros internos

Dados binários grandes vão para ficheiros em `filesDir` (persistentes) ou `cacheDir` (que o sistema pode limpar quando precisa de espaço). Guardar os bytes dentro da base de dados torna-a pesada e lenta, por isso o padrão habitual é gravar o ficheiro no disco e guardar na base de dados apenas o caminho. Isto cria uma responsabilidade: manter a linha e o ficheiro coerentes. No FixTag:

- cada anexo é gravado em `files/attachments/<id>.jpg` ou `<id>.m4a`, onde `<id>` é o identificador do anexo;

- a aplicação da câmara escreve no ficheiro através de um URI `content://` do `FileProvider`, sem acesso ao resto do armazenamento;
- os downloads são escritos num ficheiro temporário e só são renomeados quando estão completos;
- ao apagar, as linhas são removidas numa transação e só depois são apagados os ficheiros.

1.4 SQLite e Room

Para dados estruturados, o Android inclui o SQLite. Usá-lo diretamente (`SQLiteOpenHelper` e `Cursor`) obriga a escrever o SQL em texto, a converter colunas em objetos à mão e a descobrir os erros só em execução. A biblioteca **Room** é uma camada sobre o SQLite com três peças:

- **Entidade** (`@Entity`): uma classe que corresponde a uma tabela, com chave primária, chaves estrangeiras e índices. Os tipos que o SQLite não conhece passam por um `@TypeConverter`; no FixTag, `Uuid` é guardado como BLOB, `Instant` como INTEGER (milissegundos) e os enums pelo nome.
- **DAO** (`@Dao`): uma interface com as operações CRUD (`@Insert`, `@Upsert`, `@Update`, `@Delete`) e consultas `@Query`, cujo SQL é verificado durante a compilação. Uma consulta pode devolver um valor (função `suspend`) ou um `Flow`, que volta a emitir sempre que uma das tabelas lidas muda. As relações leem-se com `@Embedded` e `@Relation`, dentro de uma `@Transaction`.
- **Base de dados** (`@Database`): declara as entidades, a versão do esquema e os conversores. O código de acesso é gerado pelo KSP durante a compilação.

Gravar uma ocorrência e os seus anexos corre numa transação (`withTransaction { }`): ou fica tudo gravado, ou nada.

1.5 Migrações

Cada alteração ao esquema obriga a aumentar a versão da base de dados e a indicar à Room como converter os dados existentes, com uma `Migration` escrita à mão em SQL ou com uma `AutoMigration`, que a Room gera comparando os esquemas exportados (`exportSchema = true`; ficheiros JSON em `android/app/schemas/`, que devem estar no Git). Se não houver caminho de migração e a base de dados tiver sido configurada com `fallbackToDestructiveMigration()`, a Room apaga e recria todas as tabelas. Numa aplicação offline-first isto é grave: os registos que ainda não tinham sido enviados perdem-se sem aviso. A experiência E5 mede este efeito.

1.6 Estratégia offline-first

- **Fonte de verdade local.** A interface observa a Room através de um *repository* e nunca chama a API diretamente.
- **Estado de sincronização por registo.** Cada linha tem uma coluna `sync_status` (`Pending`, `Synced` ou `Failed`). O número de registos «por enviar» é uma consulta à base de dados.
- **Identificadores gerados no cliente.** Cada registo nasce com um UUID gerado no dispositivo, por isso pode ser criado sem servidor, e o envio (`PUT /incidents/{id}`) pode ser repetido sem criar duplicados.
- **Sincronização em segundo plano.** O WorkManager envia quando há rede (`NetworkType.CONNECTED`): primeiro os registos pendentes, depois o que mudou no servidor, com novas tentativas automáticas.
- **Conflitos.** Quando o mesmo registo muda em dois sítios, é preciso uma regra: o servidor ganha, a última escrita ganha, ou regras por campo. O FixTag evita a maioria dos conflitos: o histórico de uma ocorrência é só de acrescentar e o estado só é alterado pela assistência.

Na variante `demo`, usada neste protocolo, não há servidor: o WorkManager e a API estão desligados (`BuildConfig.OFFLINE_DEMO`) e todos os registos criados ficam `Pending`.

1.7 Alternativas em Java

O DataStore, as funções `suspend` e o `Flow` são APIs predominantemente Kotlin. Em Java usam-se as `SharedPreferences` ou o `RxDataStore` (artefacto `datastore-preferences-rxjava3`). Na Room, `LiveData` ou `Flowable` (`room-rxjava3`) substituem o `Flow`, um `Executor` ou `ListenableFuture` (`room-guava`) substituem as funções `suspend`, `runInTransaction()` substitui `withTransaction { }` e o Gradle usa `annotationProcessor` em vez de `ksp`. No WorkManager, um `Worker` substitui o `CoroutineWorker`.

2. Objetivos

Verificar, com medições objetivas, que:

- **O1**: a sessão guardada no DataStore sobrevive ao fim do processo e determina o ecrã inicial;
- **O2**: é possível criar, sem rede, uma ocorrência com foto e áudio, e que a Room e os ficheiros internos ficam coerentes;
- **O3**: as linhas e os ficheiros sobrevivem ao fim forçado do processo, e o tempo de arranque não depende da rede;
- **O4**: apagar um registo pendente remove as suas linhas e os seus ficheiros;
- **O5**: subir a versão do esquema sem migração apaga os dados por enviar, e com uma `Migration` preserva-os.

3. Configuração

3.1 Configuração utilizada

Item	Valor
Sistema anfitrião	Windows, com Android Studio e o emulador do Android SDK
Ferramentas de compilação	Android Gradle Plugin 9.4.1, KSP 2.3.4, JDK 25 (toolchain do Gradle, descarregado automaticamente)
SDK	`compileSdk` e `targetSdk` 37, `minSdk` 29
Emulador	«Medium Phone API 37.0», imagem Google Play x86_64; câmara traseira `virtualscene`; entrada de áudio ativa
Variante	`demoDebug` (`applicationId` `com.example.fixtag.demo`)
Bibliotecas	Room 2.8.5, DataStore Preferences 1.2.1, WorkManager 2.12.0 (desligado na variante demo), Coil 3.4.0
Ferramentas de observação	`adb` (Android SDK Platform-Tools); Android Studio: App Inspection → Database Inspector e Device Explorer

3.2 Requisitos para reproduzir

- Dispositivo ou emulador com Android 10 (API 29) ou superior, com câmara e microfone.
- Uma compilação `debug`: o Database Inspector e o comando `adb shell run-as` só funcionam com aplicações depuráveis.
- No emulador, em *Extended controls* → *Microphone*, ativar «Virtual microphone uses host audio input»; sem isto, a gravação de áudio pode falhar. A câmara `virtualscene` fotografa uma sala virtual.
- Dispositivo em português. Os textos citados são os da aplicação em português; em inglês aparecem as versões inglesas (por exemplo, «3 to send»).

3.3 Preparação

1. `git clone https://github.com/0xmaiev/FixTag.git` e `git checkout entrega-10-27`.
2. Abrir a pasta `android/` no Android Studio e esperar pela sincronização do Gradle.
3. Em *Build Variants*, escolher `demoDebug` para o módulo `app`.
4. Arrancar o emulador e garantir um estado limpo: `adb uninstall com.example.fixtag.demo` (o erro, se a aplicação não estiver instalada, pode ser ignorado).
5. Instalar e abrir a aplicação com *Run* (ou `./gradlew :app:installDemoDebug` dentro de `android/`).
6. Ligar o modo de voo no painel rápido ou com `adb shell cmd connectivity airplane-mode enable`.

Comandos e consultas usados nas medições:

Ação	Comando
Terminar o processo	`adb shell am force-stop com.example.fixtag.demo`
Abrir e medir o arranque	`adb shell am start -W -n com.example.fixtag.demo/com.example.fixtag.ui.main.MainActivity`
Listar os anexos	`adb shell run-as com.example.fixtag.demo ls -l files/attachments`
Ver o ficheiro do DataStore	`adb exec-out run-as com.example.fixtag.demo cat files/datastore/session.preferences_pb`
Consultar a base de dados	Android Studio → App Inspection → Database Inspector → `database` → *New query*

Consultas para o Database Inspector:

```
-- Q1: ocorrências
SELECT COUNT(*) FROM incident;
-- Q2: ocorrências por enviar
SELECT COUNT(*) FROM incident WHERE sync_status = 'Pending';
-- Q3: anexos
SELECT kind, content_type, size_bytes, sync_status, path FROM attachment;
-- Q4: total por enviar (a mesma consulta que alimenta a faixa «N por enviar»)
SELECT (SELECT COUNT(*) FROM organization WHERE sync_status = 'Pending')
     + (SELECT COUNT(*) FROM machine WHERE sync_status = 'Pending')
     + (SELECT COUNT(*) FROM incident WHERE sync_status = 'Pending')
     + (SELECT COUNT(*) FROM incident_update WHERE sync_status = 'Pending')
     + (SELECT COUNT(*) FROM attachment WHERE sync_status = 'Pending');
-- Q5: versão do esquema
PRAGMA user_version;
```

4. Dados de entrada

4.1 Dados de exemplo

São criados quando se escolhe um perfil pela primeira vez com a base de dados vazia (`android/app/src/main/java/com/example/fixtag/data/DemoSeed.kt`). Todos estão `Synced`, como se tivessem vindo do servidor; os identificadores são os mesmos dos dados de exemplo do backend.

Perfil	Nome	Organização	Ecrã inicial	`current_user_id`
Operador	Ana Operadora	Cliente Demo, Lda.	Máquinas	`b0000000-0000-4000-8000-000000000001`
Assistência	Carla Assistência	Fabricante Demo, S.A.	Ocorrências	`b0000000-0000-4000-8000-000000000003`

Nº de série	Modelo	Estado inicial	Linha dos 30 dias
FT-0001	Prensa hidráulica PH-200	«1 em aberto» (vermelho: ocorrência crítica)	«1 ocorrência nos últimos 30 dias»
FT-0002	Torno CNC TC-450	«1 em aberto» (amarelo)	«1 ocorrência nos últimos 30 dias»
FT-0003	Compressor CP-90	«OK» (verde)	«Sem ocorrências nos últimos 30 dias»

Máquina	Descrição	Gravidade	Estado
FT-0001	Fuga de óleo no cilindro principal	Crítica	Analisando
FT-0002	Ruído anormal no eixo durante o arranque	Média	Recebido

4.2 Dados introduzidos durante a experiência

Campo	Valor
Máquina	FT-0003, Compressor CP-90 (não tem ocorrências, por isso a mudança de estado é visível)
Descrição	«Teste offline: vibração anormal no motor»
Gravidade	Média (já vem selecionada)
Foto	uma, tirada com a câmara traseira
Áudio	uma gravação de cerca de 5 segundos

5. Procedimento

As experiências fazem-se por ordem, com o modo de voo ligado, a partir do estado limpo da secção 3.3. Os códigos M1 a M9 referem-se às métricas da secção 6.

E1. Sessão no DataStore (O1)

1. Abrir a aplicação. Aparece a escolha de perfil. Se for pedida a permissão de rede local, pode ser recusada: a variante demo não usa a rede.
2. Tocar em **Operador**. A aplicação abre em **Máquinas** com as três máquinas da secção 4.1.
3. Ver o ficheiro do DataStore e registar **M2**.
4. Terminar o processo, voltar a abrir a aplicação e registar **M1**.
5. Tocar em **Trocar perfil** e escolher **Assistência**. Repetir os passos 3 e 4.
6. Voltar ao perfil **Operador** (Trocar perfil → Operador) para as experiências seguintes.

E2. Criação sem rede (O2)

1. Em **Máquinas**, registar **M7** para a FT-0003 e confirmar que a faixa «N por enviar» não aparece.
2. Em **Ocorrências**, tocar em **Nova ocorrência**. Em *Máquina associada*, tocar em **Selecionar**, escolher a FT-0003 e tocar em **Selecionar**.
3. Escrever a descrição da secção 4.2 e manter a gravidade **Média**.
4. Tocar no botão da câmara, tirar uma foto e confirmá-la. Aparece a miniatura.
5. Tocar no botão do microfone, falar durante cerca de 5 segundos e tocar de novo para parar. Aparece o anexo de áudio.
6. Tocar em **Guardar**. Aparece «Ocorrência guardada com sucesso.» e a aplicação volta à lista.
7. Registar **M3** a **M7**. A M7 regista-se em **Máquinas**, sem atualizar nada à mão.

E3. Persistência após terminar o processo (O3)

1. Terminar o processo.
2. Abrir a aplicação com `am start -W` e registar o `TotalTime` (**M8**).
3. Registar **M1** e **M3** e confirmar que a ocorrência de teste está na lista.
4. Abrir a ocorrência: a descrição, a gravidade, a miniatura da foto e o anexo de áudio aparecem. Tocar na foto abre-a em ecrã inteiro; tocar no áudio reprodu-lo.
5. Registar **M4** a **M6** e comparar com os valores de E2.
6. Para a **M8**: repetir os passos 1 e 2 três vezes com o modo de voo ligado e três vezes com ele desligado, e registar a mediana de cada série.
7. Opcional: reiniciar o emulador (`adb reboot`) e repetir os passos 3 a 5.

E4. Apagar um registo pendente (O4)

1. Em **Ocorrências**, confirmar que só o cartão da ocorrência de teste tem o botão de apagar. As duas ocorrências de exemplo, já sincronizadas, não o têm.
2. Tocar no botão de apagar e confirmar em **Apagar**.

3. Registar **M3** a **M7**.

E5. Migração destrutiva e migração com `Migration` (O5)

Esta experiência altera `android/app/src/main/java/com/example/fixtag/data/Database.kt`. Correr a aplicação com *Run* atualiza-a e mantém os dados, tal como uma atualização da loja.

Parte A: sem migração

1. Criar uma ocorrência de teste com uma foto e sem áudio (passos 2 a 6 de E2). Registar **M3**, **M4**, **M5**, **M6** e **M9**.
2. Em `Database.kt`, mudar `version = 1` para `version = 2` e voltar a correr a aplicação (*Run*).
3. Registar o ecrã que aparece. Escolher **Operador** e registar **M3**, **M4**, **M5**, **M6** e **M9**.

Parte B: com migração

4. Repor o estado inicial: `git checkout -- android/app/src/main/java/com/example/fixtag/data/Database.kt`, apagar o ficheiro gerado `android/app/schemas/com.example.fixtag.data.Database/2.json`, executar `adb uninstall com.example.fixtag.demo`, voltar a correr a aplicação, escolher **Operador** e repetir o passo 1.
5. Em `Database.kt`, mudar outra vez para `version = 2` e acrescentar uma migração vazia, porque o esquema não mudou:

```
// fora da classe Database
// imports: androidx.room.migration.Migration e androidx.sqlite.db.SupportSQLiteDatabase
private val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        // o esquema não mudou, só a versão: não há dados a converter
    }
}

// em getDatabase(), no builder já existente
Room.databaseBuilder<com.example.fixtag.data.Database>(
    context.applicationContext,
    DATABASE_NAME,
).addMigrations(MIGRATION_1_2)
    .fallbackToDestructiveMigration(dropAllTables = true)
    .build()
```

6. Voltar a correr a aplicação. Registar o ecrã que aparece, **M3**, **M4**, **M5**, **M6** e **M9**.
7. Repor o código: `git checkout` do mesmo ficheiro e apagar o `2.json` gerado.

6. Métricas

Código	Métrica	Como medir
M1	Ecrã inicial depois de reabrir	Observação: escolha de perfil, Máquinas ou Ocorrências
M2	Conteúdo da sessão no DataStore	`cat` do ficheiro `session.preferences_pb` (secção 3.3): presença da chave `current_user_id` e do UUID do perfil
M3	Faixa «N por enviar»	Observação no topo das listas
M4	Ocorrências por enviar	Q2 (e Q1 para o total)
M5	Anexos na base de dados	Q3: número de linhas, `kind`, `content_type`, `size_bytes`, `sync_status`
M6	Ficheiros de anexos no disco	`ls -l files/attachments`: número, extensão e tamanho, comparados com `size_bytes` e `path` de Q3
M7	Estado da FT-0003 em Máquinas	Observação: selo («OK» ou «N em aberto») e linha dos 30 dias

Código	Métrica	Como medir
M8	Tempo de arranque a frio	`TotalTime` de `am start -W`, em milissegundos; mediana de 3 execuções
M9	Versão do esquema	Q5

7. Resultados esperados

A coluna «Obtido» é preenchida por quem reproduz a experiência.

Exp.	Métrica	Esperado	Obtido
E1	M2 (Operador)	o ficheiro contém `current_user_id` e `b0000000-0000-4000-8000-000000000001`	
E1	M1 (Operador)	Máquinas, sem escolha de perfil	
E1	M2 (Assistência)	o ficheiro contém `b0000000-0000-4000-8000-000000000003`	
E1	M1 (Assistência)	Ocorrências, sem escolha de perfil	
E2	M3	«3 por enviar» (uma ocorrência e dois anexos); Q4 = 3	
E2	M4	Q2 = 1; Q1 = 3	
E2	M5	2 linhas: `Photo` / `image/jpeg` e `Audio` / `audio/mp4`, `size_bytes` > 0, `Pending`; `path` em `files/attachments/`	
E2	M6	2 ficheiros, um `.jpg` e um `.m4a`, com os nomes e os tamanhos de Q3	
E2	M7	FT-0003 passa de «OK» para «1 em aberto» e «1 ocorrência nos últimos 30 dias», sem atualização manual	
E3	M1	Máquinas, sem escolha de perfil	
E3	M3 a M6	iguais aos valores de E2	
E3	M8	valores semelhantes com e sem modo de voo (diferença dentro da variação entre execuções): o arranque só lê dados locais	
E4	M3	a faixa desaparece; Q4 = 0	
E4	M4	Q2 = 0; Q1 = 2	
E4	M5 e M6	0 linhas em `attachment`; `files/attachments/` vazio	
E4	M7	FT-0003 volta a «OK» e «Sem ocorrências nos últimos 30 dias»	
E5A	antes	M3 «2 por enviar»; Q2 = 1; 1 anexo e 1 ficheiro; M9 = 1	
E5A	depois	aparece a escolha de perfil: a sessão no DataStore sobreviveu, mas aponta para um utilizador que já não existe na Room. Depois de escolher Operador: sem faixa; Q2 = 0; Q1 = 2 (só os dados de exemplo, recriados); 0 linhas em `attachment` mas 1 ficheiro em disco (ficou órfão); M9 = 2	
E5B	depois	abre em Máquinas, sem escolha de perfil; M3 «2 por enviar»; Q2 = 1; 1 anexo e 1 ficheiro; M9 = 2	

8. Notas

- Para recomeçar do zero: `adb uninstall com.example.fixtag.demo`, repor `Database.kt` com `git checkout` e apagar o `2.json` gerado. Os dados de exemplo só são criados quando a base de dados está vazia.

- No fim, desligar o modo de voo: `adb shell cmd connectivity airplane-mode disable`.
- O Database Inspector só mostra a base de dados enquanto a aplicação está a correr.
- Se a gravação de áudio não aparecer depois de parar, confirmar a configuração do microfone virtual (secção 3.2).

Referências

- Android Developers, *Data and file storage overview*: https://developer.android.com/training/data-storage
- Android Developers, *DataStore*: https://developer.android.com/topic/libraries/architecture/datastore
- Android Developers, *Save data in a local database using Room*: https://developer.android.com/training/data-storage/room
- Android Developers, *Migrate your Room database*: https://developer.android.com/training/data-storage/room/migrating-db-versions
- Android Developers, *Build an offline-first app*: https://developer.android.com/topic/architecture/data-layer/offline-first